

BASH SCRIPTING FUNDAMENTALS

Control Flow

Branching and looping with if, case, for, and while

Merit Advisory

Bash Scripting Fundamentals Bootcamp

Beginner-friendly lecture materials

July 2026

July 2026



Agenda

- 1 Section 1: Exit Status Fundamentals — 7 topics
- 2 Section 2: Test Commands And Strings — 7 topics
- 3 Section 3: Patterns Regex And Integers — 7 topics
- 4 Section 4: File Tests — 7 topics
- 5 Section 5: Combining Conditions — 7 topics
- 6 Section 6: Case Statements — 7 topics
- 7 Section 7: For Loops — 7 topics
- 8 Section 8: While Until And Read — 7 topics

- 9 Section 9: Pipelines Select And Arrays — 7 topics
- 1 Section 10: Validation Patterns — 7 topics
- 0
- 1 Section 11: Nested Logic Retry And Timeouts — 7 topics
- 1
- 1 Section 12: Practice And Checkpoints — 7 topics
- 2

1 Section 1

Exit Status Fundamentals

1.1 Exit status means success or failure

- Every command finishes with a numeric exit status.

Success status

```
true  
echo $?
```

- Status 0 means success in Bash conditionals.

Failure status

```
false  
echo $?
```

1.2 true is a successful command

- The true command exists only to return success.

Always succeeds

```
if true; then echo yes; fi
```

- It is useful when a loop or branch needs a placeholder.

Infinite loop header

```
while true; do break; done
```

1.3 false is a failing command

- The false command exists only to return failure.

Always fails

```
if false; then echo yes; else echo no; fi
```

- It is useful for examples and disabled branches.

Fallback

```
false || echo fallback
```

1.4 The dollar question variable

- \$? stores the most recent command status.

Capture status

```
mkdir demo  
status=$?
```

- Read it immediately before another command changes it.

Immediate print

```
test -d demo  
echo $?
```


1.5 Basic if then fi

- An if statement runs a command as its condition.

Directory check

```
if test -d /tmp; then echo exists; fi
```

- The then block runs only when the status is 0.

Command check

```
if date >/dev/null; then echo ok; fi
```

1.6 Else branch

- else handles the failed condition path.

Path missing

```
if [[ -e app.log ]]; then echo found; else echo missing; fi
```

- It runs when the if command returns nonzero.

Command failed

```
if ping -c1 host; then echo up; else echo down; fi
```

1.7 Elif chain

- elif tests another condition after a failure.

Mode branch

```
if [[ $mode == dev ]]; then echo d; elif [[ $mode == prod ]]; then echo p; fi
```

- Branches are checked from top to bottom.

Number branch

```
if (( n < 0 )); then echo neg; elif (( n == 0 )); then echo zero; fi
```

2 Section 2

Test Commands And Strings

2.1 The test command

- test evaluates file, string, and numeric expressions.

String test

```
if test -n "$name"; then echo set; fi
```

- It returns 0 for true and nonzero for false.

File test

```
test -f config.env  
echo $?
```

2.2 Single bracket test

- The `[` command is another spelling of `test`.

Bracket form

```
if [ -f config.env ]; then echo ok; fi
```

- It requires spaces after `[` and before `]`.

Required spaces

```
[ "$x" = yes ] && echo yes
```

2.3 Double bracket test

- `[[` is a Bash conditional compound command.

Safer string test

```
if [[ $name == root ]]; then echo admin; fi
```

- It avoids word splitting and pathname expansion.

Empty safe

```
[[ $value == "" ]] && echo empty
```

2.4 Empty string with -z

- The -z operator tests for zero string length.

Missing value

```
if [[ -z $user ]]; then echo missing; fi
```

- It is true when a value is empty.

Argument check

```
[[ -z $1 ]] && echo need-arg
```


2.5 Nonempty string with -n

- The -n operator tests for nonzero string length.

Value present

```
if [[ -n $token ]]; then echo set; fi
```

- It is true when a value contains at least one character.

Prompt guard

```
[[ -n $answer ]] || echo blank
```

2.6 String equality

- Use == inside [[for string equality.

Double bracket

```
[[ $env == prod ]] && echo live
```

- Use = inside [for portable test equality.

Single bracket

```
[ "$env" = prod ] && echo live
```

2.7 String inequality

- Use != to test that strings differ.

Not equal

```
if [[ $env != prod ]]; then echo safe; fi
```

- In [[, the right side can act as a pattern when unquoted.

Literal compare

```
[[ $name != "*.tmp" ]] && echo literal
```

3 Section 3

Patterns Regex And Integers

3.1 Pattern matching with double brackets

- Inside `[[, ==` can match glob patterns.

Suffix pattern

```
[[ $file == *.txt ]] && echo text
```

- Leave the pattern unquoted to enable pattern matching.

Prefix pattern

```
[[ $name == app-* ]] && echo app
```

3.2 Quoted patterns become literals

- A quoted right side of == is compared literally.

Literal star

```
[[ $value == "*.txt" ]] && echo literal
```

- This prevents glob characters from acting as patterns.

Pattern star

```
[[ $value == *.txt ]] && echo match
```

3.3 Regex match operator

- The =~ operator matches a string against a regular expression.

Digits only

```
[[ $id =~ ^[0-9]+$ ]] && echo digits
```

- The regex should usually be unquoted in [[:

Name shape

```
[[ $name =~ ^[a-z_][a-z0-9_]*$ ]] && echo ok
```

3.4 Integer equality operators

- Use -eq for integer equality in test expressions.

Equal

```
if [[ $count -eq 0 ]]; then echo empty; fi
```

- Use -ne for integer inequality.

Not equal

```
[[ $retries -ne 3 ]] && echo keep-going
```


3.5 Integer less than operators

- Use -lt to test less than.

Less than

```
[[ $age -lt 18 ]] && echo minor
```

- Use -le to test less than or equal.

At most

```
[[ $tries -le 5 ]] && echo allowed
```

3.6 Integer greater than operators

- Use -gt to test greater than.

Greater than

```
[[ $size -gt 0 ]] && echo data
```

- Use -ge to test greater than or equal.

At least

```
[[ $uid -ge 1000 ]] && echo user
```

3.7 Arithmetic conditions

- The `(( ))` form evaluates arithmetic expressions.

Counter positive

```
if (( count > 0 )); then echo yes; fi
```

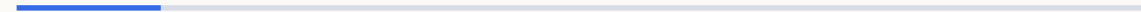
- A nonzero arithmetic result is true.

Even check

```
(( n % 2 == 0 )) && echo even
```

4 Section 4

File Tests



4.1 Path exists with -e

- The -e test checks whether a path exists.

Any path

```
[[ -e config.env ]] && echo exists
```

- It does not care what type of path it is.

Missing path

```
[[ ! -e cache ]] && echo absent
```

4.2 Regular file with -f

- The -f test checks for a regular file.

Config file

```
if [[ -f config.env ]]; then source config.env; fi
```

- It is false for directories and most special files.

Skip nonfile

```
[[ -f $path ]] || echo not-file
```

4.3 Directory with -d

- The -d test checks for a directory.

Directory exists

```
[[ -d logs ]] && echo has-logs
```

- It is useful before listing or writing inside a path.

Create if missing

```
[[ -d tmp ]] || mkdir tmp
```

4.4 Symbolic link with -L

- The -L test checks whether a path is a symbolic link.

Link check

```
[[ -L current ]] && echo symlink
```

- It is useful before replacing or resolving links.

Remove link

```
if [[ -L old ]]; then rm old; fi
```


4.5 Readable path with -r

- The -r test checks read permission for the current user.

Readable file

```
[[ -r config.env ]] && echo can-read
```

- It helps produce clearer errors before opening files.

Guard read

```
if [[ ! -r $file ]]; then echo unreadable; fi
```

4.6 Writable path with -w

- The -w test checks write permission for the current user.

Writable file

```
[[ -w report.txt ]] && echo can-write
```

- It is useful before writing files or directories.

Writable directory

```
[[ -w logs ]] || echo no-write
```

4.7 Executable path with -x

- The -x test checks execute permission.

Script executable

```
[[ -x ./deploy ]] && ./deploy
```

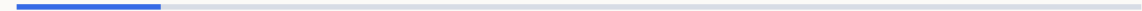
- For files, it means the file can be executed.

Directory access

```
[[ -x /tmp ]] && echo enterable
```

5 Section 5

Combining Conditions



5.1 Nonempty file with -s

- The -s test checks that a file exists and has size greater than zero.

Has content

```
[[ -s results.txt ]] && echo ready
```

- It is useful before processing generated output.

Empty report

```
[[ -s report.txt ]] || echo empty
```

5.2 And with double ampersand

- The && operator runs the right command only after success.

Two tests

```
[[ -f $file && -r $file ]] && echo usable
```

- Inside [[, && combines true conditions.

Command chain

```
mkdir out && echo made
```

5.3 Or with double pipe

- The `||` operator runs the right command only after failure.

Either suffix

```
[[ $f == *.jpg || $f == *.png ]] && echo image
```

- Inside `[[`, `||` accepts either condition.

Fallback

```
cd app || exit 1
```

5.4 Negation with exclamation

- The ! operator reverses success and failure.

Not a directory

```
if [[ ! -d logs ]]; then mkdir logs; fi
```

- It can negate commands or conditions.

Command not found

```
if ! command -v jq; then echo install-jq; fi
```


5.5 Grouping inside double brackets

- Parentheses group conditions inside `[[`.

Group alternatives

```
[[ -f $p && ( $p == *.sh || $p == *.bash ) ]] && echo shell
```

- Grouping makes precedence explicit.

Group modes

```
[[ $env == prod && ( $role == web || $role == api ) ]] && echo deploy
```

5.6 Short circuit mkdir fallback

- mkdir returns failure when it cannot create a directory.

Stop on failure

```
mkdir -p logs || exit 1
```

- A || branch can stop the script after mkdir fails.

Message then stop

```
mkdir cache || { echo failed; exit 1; }
```

5.7 Spacing mistakes in tests

- The `[` command requires separate words for each token.

Correct bracket

```
[ -n "$name" ] && echo set
```

- Missing spaces change the command name or arguments.

Correct double bracket

```
[[ -n $name ]] && echo set
```

6 Section 6

Case Statements

6.1 Case esac shape

- case compares one word against ordered patterns.

Basic case

```
case $cmd in start) echo go ;; stop) echo halt ;; esac
```

- Each matching arm runs its commands.

With variable

```
case $env in prod) echo live ;; *) echo other ;; esac
```

6.2 Exact case patterns

- A plain word pattern matches that exact value.

Mode

```
case $mode in debug) echo debug ;; esac
```

- Patterns are checked in order.

Answer

```
case $answer in yes) echo ok ;; no) echo skip ;; esac
```

6.3 Case alternatives with pipe

- The | character separates patterns in one case arm.

Yes answers

```
case $ans in y|yes) echo yes ;; n|no) echo no ;; esac
```

- The same commands run for any listed alternative.

Start aliases

```
case $cmd in run|start) echo starting ;; esac
```

6.4 Case glob patterns

- case patterns use shell glob syntax.

Suffix

```
case $file in *.sh) echo shell ;; esac
```

- A star can match any string segment.

Range

```
case $key in [0-9]) echo digit ;; esac
```


6.5 Default case arm

- The * pattern is commonly used as a default.

Unknown command

```
case $cmd in start) echo go ;; *) echo unknown ;; esac
```

- Place it last so specific patterns are checked first.

Default value

```
case $color in red) echo stop ;; *) echo pass ;; esac
```

6.6 Case arm terminator

- The ;; token ends a case arm.

Two arms

```
case $n in 1) echo one ;; 2) echo two ;; esac
```

- Bash does not fall through by default.

Default stop

```
case $x in a) echo A ;; *) echo other ;; esac
```

6.7 Menus with case

- case pairs naturally with menu choices.

Choice

```
case $choice in 1) echo add ;; 2) echo remove ;; *) echo bad ;; esac
```

- Each arm can call one action.

Letter menu

```
case $op in q) exit 0 ;; h) echo help ;; esac
```

7 Section 7

For Loops



7.1 For in word list

- A for loop assigns one word at a time to a variable.

Names

```
for name in ann bob; do echo $name; done
```

- The do block runs once for each word.

Modes

```
for mode in dev test prod; do echo $mode; done
```

7.2 For loop over quoted values

- Quoted items stay single loop values.

Quoted words

```
for item in "two words" three; do echo "$item"; done
```

- Unquoted expansions can split into multiple words.

Array values

```
items=("two words" three)  
for item in "${items[@]}"; do echo "$item"; done
```

7.3 For loop over arguments

- A for loop without in uses the script arguments.

Implicit args

```
for arg; do echo "$arg"; done
```

- Inside functions, it uses the function arguments.

Explicit args

```
for arg in "$@"; do echo "$arg"; done
```

7.4 For loop over globs

- Globs expand to matching pathnames before the loop starts.

Shell scripts

```
for f in *.sh; do echo "$f"; done
```

- Quote the loop variable inside the body.

Logs

```
for log in logs/*.log; do gzip "$log"; done
```


7.5 Empty glob care

- An unmatched glob remains literal by default.

Guard glob

```
for f in *.csv; do [[ -e $f ]] || continue; echo "$f"; done
```

- That can make a loop process a fake filename.

Use nullglob

```
shopt -s nullglob  
for f in *.csv; do echo "$f"; done
```

7.6 C style for loop

- The for ((init; test; update)) form uses arithmetic.

Count up

```
for ((i=0; i<3; i++)); do echo $i; done
```

- It is useful for numeric counters.

Count down

```
for ((i=3; i>0; i--)); do echo $i; done
```

7.7 Arithmetic step in for

- The update expression can change by more than one.

Step by two

```
for ((i=0; i<10; i+=2)); do echo $i; done
```

- Use += and -= for clear step sizes.

Halve

```
for ((n=8; n>0; n/=2)); do echo $n; done
```

8 Section 8

While Until And Read

8.1 While loop

- while repeats while its condition command succeeds.

Counter

```
i=0  
while (( i < 3 )); do echo $((i++)); done
```

- The condition is checked before each iteration.

Readiness

```
while [[ ! -f done.flag ]]; do sleep 1; done
```

8.2 Until loop

- until repeats while its condition command fails.

Wait for file

```
until [[ -f ready ]]; do sleep 1; done
```

- It stops when the condition succeeds.

Retry command

```
until curl -fsS http://localhost; do sleep 2; done
```

8.3 Break exits a loop

- break stops the nearest enclosing loop.

Find first

```
for f in *.log; do [[ -s $f ]] && { echo $f; break; }; done
```

- It is useful when a success condition appears early.

Stop menu

```
while true; do read -r x; [[ $x == q ]] && break; done
```

8.4 Continue skips to the next iteration

- continue skips the rest of the current loop body.

Skip dirs

```
for p in *; do [[ -d $p ]] && continue; echo $p; done
```

- It is useful for filtering invalid items.

Skip blanks

```
while read -r line; do [[ -z $line ]] && continue; echo $line; done
```


8.5 Infinite loops need exits

- Infinite loops are intentional only with a clear exit path.

Menu loop

```
while true; do read -r c; [[ $c == q ]] && break; done
```

- Use break, return, or exit for controlled termination.

Polling loop

```
while true; do [[ -f ready ]] && break; sleep 1; done
```

8.6 Reading lines with while read

- read gets one line of input at a time.

Read file

```
while read -r line; do echo "$line"; done < names.txt
```

- A while read loop is standard for line processing.

Number lines

```
n=0  
while read -r line; do echo $((++n)): $line; done < file
```

8.7 IFS read r

- IFS= preserves leading and trailing whitespace for read.

Safe read

```
while IFS= read -r line; do printf '%s\n' "$line"; done < file
```

- The -r option prevents backslash escapes.

Prompt read

```
IFS= read -r answer
```

9 Section 9

Pipelines Select And Arrays

9.1 Pipelines in if

- An if condition can be a pipeline.

Search output

```
if ps aux | grep -q '[s]shd'; then echo running; fi
```

- The pipeline status normally comes from the last command.

Count match

```
if printf '%s\n' a b | grep -q b; then echo found; fi
```

9.2 Pipeline while read scope

- A pipeline may run the while loop in a subshell.

Pipeline count

```
printf '%s\n' a b | while read -r x; do echo $x; done
```

- Variable changes can disappear after the loop.

Avoid counting outside

```
count=0  
printf x | while read -r _; do ((count++)); done
```

9.3 Redirected while read

- Redirecting input into a loop keeps the loop in the current shell.

Count lines

```
count=0  
while read -r _; do ((count++)); done < file
```

- This is preferred when updating variables.

Collect names

```
while read -r name; do echo "hi $name"; done < names.txt
```

9.4 Mapfile intro

- mapfile reads lines into an array.

Read array

```
mapfile -t lines < names.txt  
echo "${#lines[@]}"
```

- It is useful when later code needs random access.

First line

```
mapfile -t rows < data.txt  
echo "${rows[0]}"
```


9.5 Readarray alias

- readarray is another name for mapfile.

Readarray

```
readarray -t files < list.txt  
printf '%s\n' "${files[@]}"
```

- Both are Bash builtins in Bash 4 and later.

Count rows

```
readarray -t rows < data.txt  
echo "${#rows[@]}"
```

9.6 Select menu

- select builds a numbered menu from words.

Simple select

```
select env in dev prod quit; do echo "$env"; break; done
```

- It stores the selected item in the loop variable.

Case with select

```
select op in add quit; do case $op in quit) break ;; esac; done
```

9.7 PS3 select prompt

- PS3 controls the prompt displayed by select.

Custom prompt

```
PS3='Choose: '  
select item in one two; do echo "$item"; break; done
```

- Set it before the select loop.

Menu prompt

```
PS3='Action: '  
select op in build test; do break; done
```

10

Section 10

Validation Patterns

10.1 Validate integers with regex

- Use `=~` to confirm input is an integer before math.

Unsigned integer

```
[[ $n =~ ^[0-9]+$ ]] || echo not-int
```

- Anchors prevent partial matches.

Signed integer

```
[[ $n =~ ^-?[0-9]+$ ]] && echo int
```

10.2 Validate nonnegative counts

- Counts often require zero or positive values.

Count value

```
if [[ $count =~ ^[0-9]+$ ]]; then echo ok; fi
```

- Regex validation avoids arithmetic errors from text.

Positive value

```
[[ $n =~ ^[0-9]+$ && $n -gt 0 ]] && echo positive
```

10.3 Validate path existence

- Check that a path exists before acting on it.

Require path

```
[[ -e $path ]] || { echo "missing: $path"; exit 1; }
```

- Use -e when any path type is acceptable.

Optional path

```
if [[ -e $path ]]; then echo found; fi
```

10.4 Validate readable files

- Readable file checks combine -f and -r.

Config guard

```
[[ -f $cfg && -r $cfg ]] || exit 1
```

- This rejects directories and unreadable files.

Read branch

```
if [[ -r $file ]]; then wc -l < "$file"; fi
```


10.5 Validate writable directories

- Writable directory checks combine -d and -w.

Output directory

```
[[ -d $out && -w $out ]] || exit 1
```

- Use them before creating output files.

Create output

```
[[ -d out ]] || mkdir out
```

10.6 Validate commands with command v

- `command -v` checks whether a command can be found.

Need jq

```
if ! command -v jq >/dev/null; then echo need-jq; fi
```

- It works for builtins, functions, and executables.

Need git

```
command -v git >/dev/null || exit 1
```

10.7 Error branches

- Validation failures should take a clear branch.

Fatal branch

```
[[ -n $name ]] || { echo name-required; exit 1; }
```

- Use exit or return after fatal errors.

Nonfatal branch

```
if [[ -z $tag ]]; then tag=latest; fi
```

11

Section 11

Nested Logic Retry And Timeouts

11.1 Nested if

- Nested if statements express dependent decisions.

Nested check

```
if [[ -d repo ]]; then if [[ -f repo/config ]]; then echo ok; fi; fi
```

- Each inner test should rely on the outer branch being true.

Nested command

```
if cd repo; then if git status; then echo clean; fi; fi
```

11.2 Guard clauses reduce nesting

- A guard clause exits early when a requirement fails.

Require arg

```
[[ -n $1 ]] || exit 2  
echo "arg=$1"
```

- It keeps the main branch less indented.

Require dir

```
[[ -d src ]] || exit 1  
echo build
```

11.3 Retry loop

- A retry loop repeats a command after failure.

Three tries

```
for i in 1 2 3; do curl -fsS URL && break; done
```

- Limit retries so failures do not run forever.

Retry flag

```
ok=0  
for i in 1 2 3; do command && { ok=1; break; }; done
```

11.4 Sleep between retries

- sleep adds delay between attempts.

Sleep retry

```
for i in 1 2 3; do curl -fsS URL && break; sleep 2; done
```

- It protects services from tight retry loops.

Until sleep

```
until nc -z host 80; do sleep 1; done
```


11.5 Timeout command

- timeout limits how long a command may run.

Limit curl

```
if timeout 5 curl -fsS URL; then echo ok; fi
```

- It returns failure when the time limit is reached.

Limit read

```
if timeout 10 bash -c 'read -r x'; then echo got; fi
```

11.6 Failure limit

- Track attempts when retrying manually.

Attempt counter

```
tries=0  
until cmd; do ((++tries == 3)) && exit 1; done
```

- Stop after a known maximum failure count.

Loop bound

```
for ((i=1; i<=3; i++)); do cmd && break; done
```

11.7 Break on success

- Successful work often ends a search loop.

First match

```
for f in *.txt; do grep -q needle "$f" && { echo "$f"; break; }; done
```

- Use break immediately after the success action.

First port

```
for p in 8000 8001; do nc -z localhost $p && { echo $p; break; }; done
```

1 2

Section 12

Practice And Checkpoints

12.1 If command practice

- Use if directly with the command being tested.

Grep status

```
if grep -q root /etc/passwd; then echo found; fi
```

- Avoid checking \$? when an if command is clearer.

Git status

```
if git rev-parse --is-inside-work-tree >/dev/null 2>&1; then echo repo; fi
```

12.2 Case dispatch practice

- Use case to dispatch small command names.

Dispatch

```
case $1 in build) echo build ;; test) echo test ;; *) echo usage ;; esac
```

- Keep each arm focused on one action.

Aliases

```
case $1 in rm|delete) echo remove ;; *) echo unknown ;; esac
```

12.3 File loop practice

- Combine globs with file tests for safe batches.

Readable logs

```
for f in logs/*.log; do [[ -r $f ]] || continue; echo "$f"; done
```

- Skip nonmatching or invalid paths early.

Executable scripts

```
for f in bin/*; do [[ -x $f ]] && echo "$f"; done
```

12.4 Prompt until valid

- A while loop can repeat until input passes validation.

Ask number

```
while true; do read -r n; [[ $n =~ ^[0-9]+$ ]] && break; done
```

- Validate immediately after each read.

Ask yes no

```
while read -r a; do [[ $a == y || $a == n ]] && break; done
```


12.5 Select and case practice

- select provides the menu display.

Action menu

```
select op in build quit; do case $op in quit) break ;; build) echo build ;; esac; done
```

- case handles the selected action.

Environment menu

```
select env in dev prod; do [[ -n $env ]] && break; done
```

12.6 Missing fi mistake

- Every if statement must end with fi.

Closed if

```
if [[ -n $x ]]; then echo set; fi
```

- Missing fi often reports a syntax error later in the file.

Nested closed

```
if [[ -d a ]]; then if [[ -f a/b ]]; then echo ok; fi; fi
```

12.7 Control flow checkpoint

- Choose if for command status and simple decisions.

Decision

```
[[ -f config ]] && echo file || echo missing
```

- Choose case for one value matched against many patterns.

Loop

```
for f in *.sh; do bash -n "$f"; done
```

NEXT STEP

Practice every example in your terminal

Then open the next module deck

Merit Advisory
03 · Control Flow

